OAuth no AsNet — o mapa do fluxo

A tese em uma frase: o OAuth responde exatamente a *uma* pergunta — «quem é esta pessoa?» — e termina no callback. Tudo o que acontece a seguir (empresa, subscritor, cookie, gate de domínios) é código nosso, e é lá que o T-0036 vai viver. Companheiro deste documento: **OAuth_From_Scratch_Guide.pdf** (como construir isto de raiz).

1. O elenco

A pessoa

Dona da identidade. A password dela nunca passa pelo AsNet.

O browser

O estafeta — transporta redirects e o «voucher», nunca segredos.

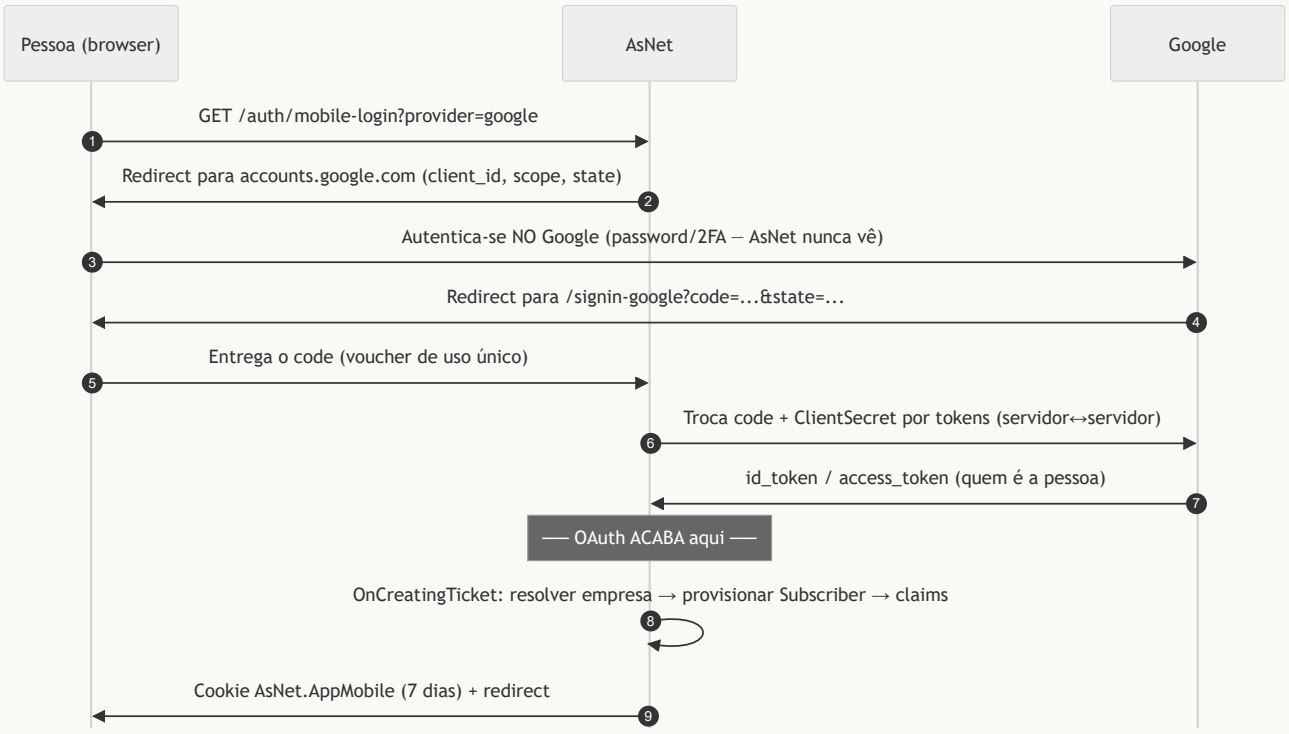
AsNet (a app)

O «cliente» OAuth. Tem um `ClientId` público e um `ClientSecret` que só o servidor conhece.

Google / Microsoft

O fornecedor de identidade. Verifica a pessoa e emite provas assinadas.

2. A dança (authorization code flow)



1

O empurrão

A app pede um `Challenge` ao esquema Google.

`AppMobileLoginController.cs:74` — GET /auth/mobile-login

2

O convite

O handler do ASP.NET constrói o URL de autorização: quem pergunta (`client_id`), o que quer saber (`scope: openid email profile`), para onde voltar (`redirect_uri`) e um `state` anti-forja.

Handler registado em `Program.cs:573` (AddGoogle) / `:665` (AddOpenIdConnect)

3

A prova

A pessoa autentica-se *no Google*. É este o truque todo: a password nunca toca no AsNet.

4

O voucher

O Google devolve o browser com um `code` — um talão de uso único, inútil para quem o apanhar: sem o `ClientSecret` não compra nada.

CallbackPath `/signin-google` — intercetado pelo middleware ANTES de qualquer controller

5

A troca

Canal servidor↔servidor: `code + ClientSecret → tokens`. O browser nunca vê tokens.

6

A identidade

O handler valida e constrói o `ClaimsPrincipal`: email, nome, id externo.
O OAuth termina aqui.

7

A nossa parte

Os eventos de enriquecimento correm código AsNet: resolver a empresa a partir do domínio do email, provisionar o `Subscriber`, juntar claims próprias (`UserType`, `SubscriberId`, `CompanyDatabase`) e assinar o cookie `AsNet.AppMobile`.

`Program.cs:595` OnCreatingTicket (Google) · `:695` OnTokenValidated (Microsoft) · `AppMobileCompanyResolver.cs` · `MobileSubscriberProvisioningService.cs`

ONDE ENTRA O T-0036

O gate de domínios **já existe** — em `AppMobileCompanyResolver.ResolveForExternalEmailAsync`: domínio fora de `AppMobile:EmailDomainMappings` → login recusado. O problema é a fonte: é *configuração no servidor*, não a tabela `EntitySiteDomain` que a ficha da entidade gere — e hoje o mapeamento inclui `gmail.com → TestCompany`, ou seja, qualquer Gmail do planeta entra.

A mudança: o resolver passa a perguntar à `EntitySiteDomain` (por empresa) em vez de/além do config. O OAuth não é tocado — mudamos a resposta a «esta identidade é bem-vinda?», nunca a «quem é?».

3. Utilizadores internos — sem OAuth

Username + password contra a nossa BD (BCrypt), as mesmas claims com `UserType=User`, o mesmo cookie. Técnicos saltam para `/tecnico` via a ponte de sessão. (`AppMobileLoginController.cs:86` · `TechnicianSessionController.cs`)

4. Achado da visita guiada — código morto

NO CONTROLLER	ESTADO
`BuildGoogleAuthorizationUrl` / `BuildMicrosoftAuthorizationUrl` / `CreateState`	Sem chamadores — geração anterior, morta
`GoogleCallback` / `MicrosoftCallback` / `HandleCallbackAsync` / `SaveSubscriberAsync`	Sombreados: o middleware consome os CallbackPaths antes do routing — nunca correm
`AppMobileEmailAuthorizationService.IsEmailAllowed`	Registado, nunca invocado — `AllowedEmailDomains` só afina mensagens de erro

Limpeza natural dentro do T-0036 — remover a geração morta torna o fluxo real óbvio para o próximo leitor. · Documento gerado a 2026-09-04 a partir do código em AsNet2.0 master.